

Guía de trabajo en Jupyter Notebook para Análisis Numérico

Estructura recomendada de un cuaderno

Cada problema o ejercicio debe resolverse siguiendo un flujo ordenado que combine celdas Markdown (para explicaciones, fórmulas y análisis) y celdas Code (para implementación y cálculos). Plantilla tipo:

Título del problema

Enunciado

(Reproducir el enunciado original en una celda Markdown)

Análisis teórico

(Derivar ecuaciones, explicar métodos, justificar intervalos, etc.)

Implementación

(Código Python con funciones, importaciones y llamadas a los métodos)

Resultados

(Imprimir tablas, gráficas, comparativas)

Verificación y conclusiones

(Comparar con valores esperados, discutir errores, limitaciones)

Buenas prácticas

1. Separar claramente la teoría del código:

- Usar celdas Markdown para todo lo que no sea código ejecutable.
- Las fórmulas matemáticas deben ir dentro de $\$ \dots \$$ (inline) o $\$ \$ \dots \$ \$$ (ecuación en bloque).
- Evitar el uso de $\backslash left$ y $\backslash right$ dentro de exponentes o subíndices, ya que pueden causar errores de renderizado.

2. Estructura de las celdas de código:

- Importar todas las librerías necesarias al inicio (numpy, matplotlib, scipy, etc.).
- Definir las funciones f , df , ddf , etc. en celdas separadas o agrupadas lógicamente.
- Usar nombres descriptivos ($f_{ejercicio1a}$, $df_{ejercicio1a}$).
- Comentar el código cuando sea necesario.

3. Reutilización de funciones propias:

- Si se han desarrollado funciones en módulos externos (como `src.root_finding`), importarlas.
- Documentar brevemente los parámetros y retornos de las funciones.

4. Visualización de resultados:

- Mostrar siempre la raíz aproximada, el número de iteraciones y la razón de parada.
- Incluir gráficas de la función para visualizar la raíz (marcar el punto).
- Usar `plt.axhline(0, color='red')` para el eje x, y ejes en colores que destaquen.

5. Manejo de tolerancias y criterios de parada:

- Definir explícitamente las tolerancias (`tol_abs`, `rel_tol`).
- Comparar los resultados con los obtenidos mediante `scipy.optimize` si es posible.

- Aceptar que la razón de parada puede ser $|f(p)| \approx 0$ en lugar de Error absoluto; adaptar las pruebas en consecuencia.

6. Control de versiones (Git):

- Realizar commits frecuentes después de resolver cada problema o grupo de problemas.
- Usar mensajes claros que describan los cambios (ej: "resuelve ejercicio 5a con Newton y modificado").
- Mantener un archivo .gitignore que excluya archivos temporales (__pycache__, .ipynb_checkpoints).

7. Pruebas unitarias (para funciones reutilizables):

- Si se escriben funciones que se usarán en varios cuadernos, almacenarlas en módulos .py y añadir pruebas con pytest.
- Verificar convergencia, manejo de errores y casos extremos.

8. Reproducibilidad:

- Fijar la semilla aleatoria si se usan números aleatorios (np.random.seed(42)).
- Especificar las versiones de las librerías en un archivo environment.yml o requirements.txt.

9. Limpieza y organización:

- Al final del cuaderno, reiniciar el kernel y ejecutar todas las celdas para asegurar que el orden es correcto.
- Eliminar celdas innecesarias o comentarios que no aporten.

Ejemplo de una celda Markdown bien formateada

Ejercicio 1a – Método de Newton

Función: $f(x) = x^2 - 2xe^{-x} + e^{-2x} = (x - e^{-x})^2$

Derivada: $f'(x) = 2x - 2e^{-x} + 2xe^{-x} - 2e^{-2x}$

Derivada segunda: $f''(x) = 2 + 2e^{-x} - 2xe^{-x} + 4e^{-2x}$

Aproximación inicial: $x_0 = 0.5$

Tolerancia: 10^{-8}

Ejemplo de una celda de código limpia

```
import math
from src.root_finding import newton, newton_modificado

def f(x):
    return (x - math.exp(-x))**2

def df(x):
    return 2*(x - math.exp(-x))*(1 + math.exp(-x))

def ddf(x):
    return 2*(1 + math.exp(-x))**2 + 2*(x - math.exp(-x))*(-math.exp(-x))
```

```
x0 = 0.5  
tol = 1e-8
```

```
print("Newton estándar:")  
raiz_n, hist_n, razon_n = newton(f, df, x0, tol_abs=tol, verbose=True)  
print(f"Raíz = {raiz_n:.8f}, iter = {len(hist_n)-1}")
```

```
print("\nNewton modificado:")  
raiz_m, hist_m, razon_m = newton_modificado(f, df, ddf, x0, tol_abs=tol, verbose=True)  
print(f"Raíz = {raiz_m:.8f}, iter = {len(hist_m)-1}")
```

Recomendaciones finales

- Nombrar el cuaderno de forma descriptiva: Ejercicios_seccion2_4.ipynb.
- Exportar a HTML o PDF cuando se quiera compartir los resultados sin necesidad de ejecutar.
- Revisar la ortografía y las fórmulas antes de entregar.
- Si se trabaja en equipo, usar ramas de Git para cada sección y hacer merge al final.